UNIVERSITY OF PERADENIYA
DEPARTMENT OF STATISTICS AND COMPUTER SCIENCE
END SEMESTER EXAMINATION - SEMESTER I (2021/2022)
CSC1051/CS105 - PROGRAMMING LABORATORY II

Answer **ALL** Questions

Time Allowed: **Three** Hours

Create a folder named CSC1051_End_S20XXX in your H drive. Create a new Java project named S20XXX. Create two packages named Q1 and Q2 inside your Java project. Implement the answer for Question 1 inside the Q1 package and implement the answer for Question 2 in the Q2 package. Make sure to save all your work inside the CSC1051_End_S20XXX folder that you created.

(Mark allocation: Question 1 = 70, Question 2 = 30)

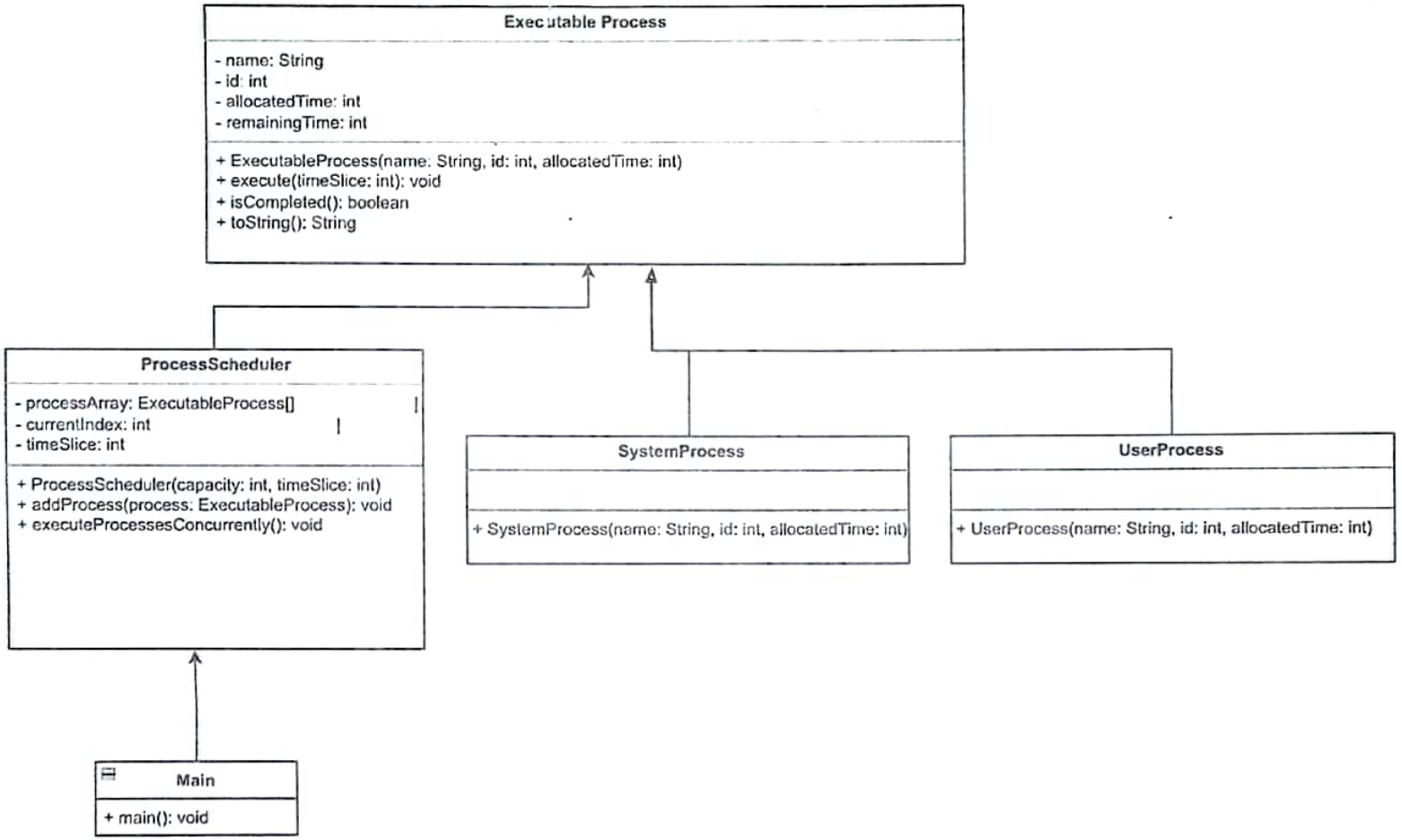
1. You are required to construct a process scheduler for an operating system that performs process scheduling, time slicing, and process execution. The scheduler must execute processes in a round-robin fashion and guarantee that each process receives an equal amount of CPU time. The necessary implementation steps for the process scheduler are described in the sections that follow.

   Please note that you will be given marks for using OOP concepts and best programming practices.

   a. *ExecutableProcess* Class is the abstract base class for all processes in the system which defines common properties and behaviors shared by all types of processes. Create the *ExecutableProcess* class with the following class definition.

Attributes
name *id* *allocatedTime* (Integer value to keep the total time the process needs to complete execution). *remainingTime* (Integer value)
Methods
public ExecutableProcess(String name, int id, int allocatedTime) *public void execute(int timeSlice)* *public boolean isCompleted()* *public String toString()* *Getter methods for all the attributes*

   b. The **public ExecutableProcess(String name, int id, int allocatedTime)** constructor should perform the following tasks.
      i. Accept and set values for all the attributes of the class correctly.


ExecutableProcess
- name: String
- id: int
- allocatedTime: int
- remainingTime: int
+ ExecutableProcess(name: String, id: int, allocatedTime: int)
+ execute(timeSlice: int): void
+ isCompleted(): boolean
+ toString(): String
ProcessScheduler
- processArray: ExecutableProcess[]
- currentIndex: int
- timeSlice: int
+ ProcessScheduler(capacity: int, timeSlice: int)
+ addProcess(process: ExecutableProcess): void
+ executeProcessesConcurrently(): void
SystemProcess
+ SystemProcess(name: String, id: int, allocatedTime: int)
UserProcess
+ UserProcess(name: String, id: int, allocatedTime: int)
Main
+ main(): void

c. The **public void execute(int timeSlice)** method should perform the following tasks.
   i. Execute the process for a period of time defined by the time slice. A process is only permitted to run for a time frame specified by the time slice, and if the allocated time is greater than the time frame specified by the time slice, the process will execute only for a period of time slice. If the allocated time is less than or equal to the time frame specified by the time slice, the process will complete execution within the time slice.

   The process execution is shown by printing a console output. Consider a system process with the name *SystemProcess1* with allocated time 5 and time slice 2. The console output for the execution of *SystemProcess1* will be as follows.

```
>>> SystemProcess1 is being executed <<<
Allocated time for process SystemProcess1 : 5
Execution time for process SystemProcess1 : 2
Remaining time for process SystemProcess1 : 3
```

d. The **public boolean isCompleted()** should perform the following tasks.
   i. Return true if the remaining process execution time is less than or equal to 0.

e. The **public String toString()** should perform the following tasks.
   i. Override the *toString* method to return the current attribute state of the *ExecutableProcess* class.

f. Create the *UserProcess* Class. The *UserProcess* is a concrete subclass of the *ExecutableProcess* class. Instances of *UserProcess* class represent processes categorized as user-level processes.
   i. The constructor of the *UserProcess* class should initialize the properties of the *UserProcess* class inherited from the *ExecutableProcess* class.

g. Create the *SystemProcess* Class. The *SystemProcess* is another concrete subclass of the *ExecutableProcess* class. Instances of *SystemProcess* class represent processes categorized as system-level processes.
   i. The constructor of the *SystemProcess* Class should initialize the properties of the *SystemProcess* class inherited from the *ExecutableProcess* class.

h. *ProcessScheduler* class is responsible for managing and executing an array of *ExecutableProcess* objects concurrently using a specified time slice. Create the *ProcessScheduler* class with the following class definition.

Attributes
processArray - an array of *ExecutableProcess* instances. *currentIndex* - variable to keep track of the current number of processes. *timeSlice* - Integer value to represent the maximum amount of time that a process can execute before the scheduler moves on to the next process.

Methods
public ProcessScheduler(int capacity, int timeSlice) *public void addProcess(ExecutableProcess process)* *public void executeProcessesConcurrently()*

i. The **public ProcessScheduler(int capacity, int timeSlice)** constructor should perform the following tasks.
   i. Take in array capacity and time slice as input parameters and initialize the processArray with the specified capacity and the timeSlice.
   ii. Initialize the currentIndex to 0.

j. The **public void addProcess(ExecutableProcess process)** method should perform the following tasks.
   i. Add *ExecutableProcess* instances to the *processArray*.
   ii. If the user attempts to add new processes after the array is full, the error message below must be displayed in the console.

```
>>> Process scheduler is full. Cannot add more processes <<<
```

k. The **public void executeProcessesConcurrently()** method should perform the following tasks.
   i. Concurrent execution of the collection of processes in the *processArray* using a round-robin scheduling approach that executes each process a fixed time slice and moves to the next process in a cyclic order. It repeatedly executes each process for a specified time frame(*timeSlice*), checks the completion status of each process, and continues the execution until all processes have been completed.
   ii. Once a process has finished executing, display an execution complete message. For example, for a system process with the name *SystemProcess1*, the message should look like below.

```
>>> Process SystemProcess1 completed <<<
```

l. Create a *Main/Driver* class and do the following tasks.
   i. Create a *ProcessScheduler* object with capacity 3 and *timeSlice* 2.
   ii. Create 3 processes with the following values for attributes.

Process type	Process name	Process id	Allocated time
System Process	SystemProcess1	1	5
System Process	SystemProcess2	2	3
User Process	UserProcess1	3	4

   iii. Call the *addProcess* method and add the above created 3 processes to the process array of the *ProcessScheduler* object.

iv. Call the *executeProcessesConcurrently* method of the *ProcessScheduler* object.

2. You are tasked with creating a Java program that performs operations on an array of integers and writes the results to a file.

Please note that you will get marks for Exception handling and best programming practices.

a. Create an array of integers with the following values: 10, 20, 30, 40, 50.

b. Implement the following methods for the program.
   i. **calculateSum(int[] numbers):** Calculate and return the sum of the integers in the array.
   ii. **calculateAverage(int[] numbers):** Calculate and return the average of the integers in the array.
   iii. **writeResultToFile(String fileName, String result):** Write the given result string to a file with the specified file name.

c. Display a menu to the user with the following options. (Please note that this is not a recurrent menu. The user will be asked once to choose whether he/she needs to calculate the sum or the average). The sample output is shown in Figure 2.1
   1. Calculate and display the sum of the integers.
   2. Calculate and display the average of the integers.

```
Menu:
1. Calculate and display the sum of the integers.
2. Calculate and display the average of the
integers.

Enter your choice (1 or 2): 1

Sum of the integers: 150
Result written to 'results.txt'
```

Figure 2.1

d. Allow the user to enter their choice (1 or 2) and perform the corresponding operation.

- If the user selects option 1, calculate the sum of the integers and display it. Then, write the result to a file named "results.txt".
- If the user selects option 2, calculate the average of the integers and display it. Then, write the result to the same file "results.txt".

—End of Paper—